

claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-a715c1965562f3f40.jsonl`
- SHA-256: `6de189c8e134528bbc8be3960e10d9e35aaafbfd6fd8785fb1e78b570e1e17a0`
- Source modified: `2026-06-21T19:13:58+00:00`
- Imported at: `2026-07-08T15:59:55+00:00`
- project: `wf_b8ec0894-3cb`
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`
```

Transcript

1. user (2026-06-21T19:12:53.045Z)

You are verifying ONE claim in khelsutra's deploy runbook (deploy/DEPLOY_ALPHA.md) against the REAL code the owner will deploy.

CRITICAL repo rules:

```
- Engine repo is at /c/Users/avidu/Projects/badminton-highlight-indexer. Its working tree is checked out on a STALE concurrent-session branch (12 commits behind master). DO NOT read the working tree for engine claims and DO NOT checkout/commit/modify anything there ? it is a shared checkout owned by another session.
- To read what the owner actually deploys, read ENGINE origin/master ONLY, via:
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer grep -n "PATTERN" origin/master
-- 'PATHGLOB'          (search master's tree)
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer show origin/master:PATH
(read a master file)
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer log origin/master --oneline -i
--grep="..."         (history)
  (I already ran 'git fetch' so origin/master is current. Never fetch/checkout.)
- khelsutra repo is at /c/Users/avidu/Projects/khelsutra, working tree is on clean master ? you MAY use Read/Grep/Glob directly there.
```

Be adversarial and literal: confirm the claim is TRUE for origin/master, with file:line evidence. If the doc says X but the code says Y, that is a 'discrepancy' ? capture it precisely. If something the owner must DO (beyond the doc's text) is implied, put it in owner_action_note. Verdict 'confirmed' only with concrete evidence; 'partial' if mostly true with a caveat; 'refuted' if wrong; 'unknown' if you genuinely could not determine it.

CLAIM (§0/§2.2): the engine ALSO verifies the Cf-Access JWT in-app (defence in depth) so a direct tunnel hit can't spoof identity; gated by security.edge_auth_enabled (engine M9 #290). On origin/master, find the edge-auth middleware/dependency (search 'cf_access\|Cf-Access\|JWKS\|jwt\|edge_auth' in backend). Confirm it does REAL signature verification: fetches JWKS from cf_team_domain, validates signature + audience (cf_access_aud) + expiry ? NOT merely checking the header is present. Note what happens when the header is missing/invalid (401?) and confirm it's a no-op when edge_auth_enabled is false. Flag any weakness (e.g. unverified alg, no aud check, JWKS not cached/validated).

2. assistant (2026-06-21T19:12:55.422Z)

I'll verify this claim about the engine's in-app Cf-Access JWT verification. Let me search origin/master for the edge-auth implementation.

3. user (2026-06-21T19:12:57.599Z)

```
origin/master:backend/api/auth.py
origin/master:backend/config/models.py
```

```
origin/master:backend/config/startup.py
origin/master:backend/main.py
origin/master:backend/utils/redact.py
```

4. user (2026-06-21T19:12:58.739Z)

```
ca34787 feat(pkg): P2b ? friendly `indexer --app` launcher (boot localhost server + open the
bundled UI) (#303)
3895963 feat(pkg): P0c ? config/profile-driven bind address + DNS-rebinding Host-header guard
(default-OFF) (#296)
fc78fce feat(pkg): P0b ? redact secret-shaped fields from the unauthenticated GET /api/config
(#295)
0c0e064 feat(serving): exposure-safety boot posture ? fail fast on a half-configured exposed
engine (default-OFF) (#294)
2afb9df fix(dryrun): install the watermark font in bootstrap + track the UI-trigger asks (#292)
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)
49c082a docs(serving): lock the 5 M0 gating decisions (D7-D12) + add the data-flywheel harness
(M11) (#277)
```

5. assistant (2026-06-21T19:13:00.973Z)

Let me read the auth implementation and config models from origin/master.

6. user (2026-06-21T19:13:02.308Z)

```
"""Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant `user_id` (M9,
D9).

**DEFAULT-OFF:** when `security.edge_auth_enabled` is False the resolver is a strict NO-OP
(returns
`None` = the local single-user) ? nothing about the existing local path changes. When enabled,
the
`Cf-Access-Jwt-Assertion` header is **required** and **verified** (RS256) against the team
JWKS ?
signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't spoof** the
identity header (the explicit D9 anti-spoof requirement). Returns the verified `user_id` (the
token's `email`, else `sub`).

`jwt` (`PyJWT[crypto]`) is **lazy-imported** so the base runtime works without it (install
the
`auth` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole verifier
is
unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no Cloudflare.
"""
```

```
from __future__ import annotations
```

```
import json
import time
import urllib.request
from typing import Any, Callable, Mapping, Optional
```

Cloudflare Access sends the signed assertion in this header (it terminates at the CF e

```
CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
_JWKS_PATH = "/cdn-cgi/access/certs"
_JWKS_TTL_SEC = 600.0 # re-fetch the team JWKS at most every 10 min (keys rotate slowly)
```

```
JwksFetcher = Callable[[str], dict]
```

```
class EdgeAuthError(Exception):
```

```
    """A required Cf-Access token was missing or failed verification ? the caller returns
401/403."""
```

module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only; t

```
_jwks_cache: dict[str, tuple[float, dict]] = {}
```

```
def _issuer(team_domain: str) -> str:
    return team_domain.rstrip("/")
```

```
def _default_jwks_fetcher(team_domain: str) -> dict:
    """Fetch the team's public JWKS (`<team_domain>/cdn-cgi/access/certs`). Network ? only the
    production path reaches here; tests pass an injected fetcher."""
    url = _issuer(team_domain) + _JWKS_PATH
    with urllib.request.urlopen(url, timeout=5) as resp: # noqa: S310 (https team domain)
        data: Any = json.load(resp)
    if not isinstance(data, dict) or "keys" not in data:
        raise EdgeAuthError(f"unexpected JWKS shape from {url}")
    return data
```

```
def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
    """Return the JWKS for `team_domain`, cached with a TTL. An injected `fetcher` (tests)
    bypasses the cache so each test is deterministic."""
    if fetcher is not None:
        return fetcher(team_domain)
    now = time.time()
    cached = _jwks_cache.get(team_domain)
    if cached and (now - cached[0]) < _JWKS_TTL_SEC:
        return cached[1]
    jwks = _default_jwks_fetcher(team_domain)
    _jwks_cache[team_domain] = (now, jwks)
    return jwks
```

```
def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) -> str:
    """Verify a Cf-Access JWT against `jwks` (RS256; checks sig + exp + issuer + audience) and
    return the user_id (`email` else `sub`). Raises `EdgeAuthError` on ANY failure."""
    try:
        import jwt
        from jwt import PyJWKSet
    except Exception as exc: # pragma: no cover - only when the auth extra isn't installed
        raise EdgeAuthError(
            "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
            "(pip install -e .[auth]) on a host with edge_auth_enabled."
        ) from exc

    try:
        kid = jwt.get_unverified_header(token).get("kid")
        if not kid:
            raise EdgeAuthError("token header has no kid")
        signing_key = PyJWKSet.from_dict(jwks)[kid]
        claims = jwt.decode(
            token, signing_key.key, algorithms=["RS256"],
            audience=aud, issuer=_issuer(team_domain),
            options={"require": ["exp", "iss", "aud"]},
        )
    except EdgeAuthError:
        raise
    except KeyError as exc: # kid not in the JWKS
        raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
    except Exception as exc: # jwt.* errors: bad sig / expired / wrong aud|iss / malformed
        raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc

    user = claims.get("email") or claims.get("sub")
    if not user:
        raise EdgeAuthError("verified token carries no email/sub claim")
```

```

return str(user)

def resolve_tenant(headers: Mapping[str, str], config: Any, *,
                  jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
    """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.

    DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-user;
    unchanged).
    Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403) on any
    miss.
    ``jwks_fetcher`` is injectable for offline tests.
    """
    sec = getattr(config, "security", None)
    if not sec or not getattr(sec, "edge_auth_enabled", False):
        return None # DEFAULT-OFF ? no tenant identity, byte-identical to today

    # Strip so a whitespace-only value counts as unset ? matching the boot-time exposure check
    in
    # config/startup.py (EXPOSED_EDGE_AUTH_INCOMPLETE); one semantics for "configured", both
    places.
    team_domain = (getattr(sec, "cf_team_domain", "") or "").strip()
    aud = (getattr(sec, "cf_access_aud", "") or "").strip()
    if not team_domain or not aud:
        raise EdgeAuthError(
            "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not configured")

    # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but support a
    plain
    # dict too (tests) by trying both casings.
    token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
    if not token:
        raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass Cloudflare
    Access)")

    jwks = _get_jwks(team_domain, jwks_fetcher)
    return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)

```

7. assistant (2026-06-21T19:13:05.631Z)

This is a thorough implementation. Now let me check how it's wired into main.py (the actual request path and 401 handling), the config model, and the deploy doc claim itself.

8. user (2026-06-21T19:13:08.274Z)

```

origin/master:backend/main.py:31:from fastapi import FastAPI, HTTPException, Request
origin/master:backend/main.py:60:from backend.api import auth as edge_auth # M9: Cloudflare
Access edge-auth (default-OFF)
origin/master:backend/main.py:117:    when an edge-auth profile is active
(``security.edge_auth_enabled`` ? i.e. there IS an auth gate
origin/master:backend/main.py:126:    if getattr(sec, "edge_auth_enabled", False):
origin/master:backend/main.py:239:from backend.api.job_worker import ( # noqa: F401 (re-
exports: names stay addressable on backend.main)
origin/master:backend/main.py:425:        raise HTTPException(status_code=400, detail=str(e))
from e
origin/master:backend/main.py:431:        raise HTTPException(status_code=400, detail=str(e))
from e
origin/master:backend/main.py:433:        raise HTTPException(status_code=404, detail=f"No
compiled reel named '{name}'.")
origin/master:backend/main.py:567:    # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ?
owner_id None (the local single-user,
origin/master:backend/main.py:568:    # unchanged). When edge_auth_enabled, the Cf-Access JWT is
required + verified (a missing or
origin/master:backend/main.py:569:    # spoofed identity header fails here with 401, before any
work) and tags the job's owner_id.

```

```

origin/master:backend/main.py:571:         owner_id = edge_auth.resolve_tenant(request.headers,
global_config)
origin/master:backend/main.py:572:     except edge_auth.EdgeAuthError as e:
origin/master:backend/main.py:573:         raise HTTPException(status_code=401, detail=f"edge
auth: {e}") from e
origin/master:backend/main.py:582:         raise HTTPException(status_code=400, detail=str(e))
from e
origin/master:backend/main.py:590:         raise HTTPException(status_code=404, detail=f"Video
file not found at '{req.video_path}')"
origin/master:backend/main.py:593:         raise HTTPException(
origin/master:backend/main.py:600:             req.player_pool,
req.max_frames, owner_id=owner_id,
origin/master:backend/main.py:602:         raise HTTPException(status_code=500, detail="Failed to
persist job record.")
origin/master:backend/main.py:617:         raise HTTPException(status_code=404, detail=f"Unknown
job_id '{job_id}')"
origin/master:backend/main.py:642:         raise HTTPException(status_code=404, detail=f"Unknown
job_id '{job_id}')"
origin/master:backend/main.py:677:         raise HTTPException(status_code=400, detail=str(e))
from e
origin/master:backend/main.py:679:         raise HTTPException(status_code=404, detail="Indexed
video record not found.")
origin/master:backend/main.py:685:         raise HTTPException(status_code=400, detail=str(e))
from e
origin/master:backend/main.py:687:         raise HTTPException(status_code=404,
origin/master:backend/main.py:708:         raise HTTPException(status_code=404, detail="Indexed
video record not found.")
origin/master:backend/main.py:715:         raise HTTPException(status_code=400, detail="No
matching play segments found to stitch.")
origin/master:backend/main.py:722:         raise HTTPException(status_code=400, detail=str(e))
from e
origin/master:backend/main.py:743:         raise HTTPException(status_code=400,
origin/master:backend/main.py:770:         raise HTTPException(status_code=500,
detail=f"Highlight stitching failed: {str(e)}") from e
origin/master:backend/main.py:873:         "security.edge_auth_enabled.
Expose on a network ONLY behind an auth gate.")
origin/master:backend/main.py:1085:         f"gate (edge_auth_enabled / a reverse
proxy) is in front, and set "

```

9. user (2026-06-21T19:13:09.148Z)

```

origin/master:backend/config/models.py:18:class ValidationRelevanceConfig(BaseModel):
origin/master:backend/config/models.py:24:class ValidationConfig(BaseModel):
origin/master:backend/config/models.py:33:class TrackNetConfig(BaseModel):
origin/master:backend/config/models.py:54:class WasbIndexConfig(BaseModel):
origin/master:backend/config/models.py:94:class FusionConfig(BaseModel):
origin/master:backend/config/models.py:127:class IndexingConfig(BaseModel):
origin/master:backend/config/models.py:230:class OutputConfig(BaseModel):
origin/master:backend/config/models.py:238:class WatermarkConfig(BaseModel):
origin/master:backend/config/models.py:255:class StitchingConfig(BaseModel):
origin/master:backend/config/models.py:271:class LoggingConfig(BaseModel):
origin/master:backend/config/models.py:280:class SecurityConfig(BaseModel):
origin/master:backend/config/models.py:297:    # DEFAULT-OFF: edge_auth_enabled=False ? no JWT
is required/verified (the local single-user
origin/master:backend/config/models.py:303:    edge_auth_enabled: bool = False
origin/master:backend/config/models.py:304:    cf_team_domain: str = "" # e.g.
https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
origin/master:backend/config/models.py:305:    cf_access_aud: str = "" # the CF Access
application AUD tag the token must carry
origin/master:backend/config/models.py:311:class StorageConfig(BaseModel):
origin/master:backend/config/models.py:348:class DeploymentConfig(BaseModel):
origin/master:backend/config/models.py:368:class BillingConfig(BaseModel):
origin/master:backend/config/models.py:387:class AppConfig(BaseModel):

```

10. assistant (2026-06-21T19:13:11.976Z)

The wiring is in place. Now let me read the SecurityConfig model fields fully, the deploy doc claim text, and check the test coverage to confirm the verification is real (not stubbed).

11. user (2026-06-21T19:13:13.715Z)

```
class SecurityConfig(BaseModel):
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
    max_upload_mb: int = 4096
    max_part_mb: int = 95
    allowed_upload_extensions: List[str] = ["mp4", "mov"]

    # --- M9 edge auth (Cloudflare Access, D9) ---
    # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
    # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-
    Assertion`
    # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
    identity
    # header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
    # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF
    Access
    # application's audience tag. NO secrets here ? these are public identifiers.
    edge_auth_enabled: bool = False
    cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs
    JWKS)
    cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
    # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at import in
    # backend/main.py so importing the app does no filesystem I/O; default [""]. Credentials
    stay
    # OFF ? Cf-Access is a header, not a cookie.)
```

12. user (2026-06-21T19:13:15.224Z)

origin/master:tests/test_edge_auth.py

13. assistant (2026-06-21T19:13:17.761Z)

Now let me verify the test coverage proves real signature verification (bad sig / expired / wrong aud rejected), and read the deploy doc claim verbatim.

14. user (2026-06-21T19:13:20.378Z)

```
3:Fully OFFLINE: an in-test RSA keypair signs Cf-Access JWTs + builds a fake JWKS, so the
verifier +
15:from jwt.algorithms import RSAAlgorithm # noqa: E402
22:AUD = "test-aud-tag"
31:    jwk.update({"kid": KID, "alg": "RS256", "use": "sig"})
35:def _token(key, *, email="u@example.com", aud=AUD, iss=TEAM, exp_delta=3600, kid=KID,
sub=None):
37:    claims = {"aud": aud, "iss": iss, "exp": now + exp_delta, "iat": now}
42:    return jwt.encode(claims, key, algorithm="RS256", headers={"kid": kid})
45:def _cfg(enabled, *, team=TEAM, aud=AUD):
47:    "edge_auth_enabled": enabled, "cf_team_domain": team, "cf_access_aud": aud}})
52:def test_valid_token_returns_email(keypair):
54:    assert edge_auth.verify_cf_access_token(
```

```

55:         _token(key), team_domain=TEAM, aud=AUD, jwks=jwks) == "u@example.com"
58: def test_falls_back_to_sub_when_no_email(keypair):
61:     assert edge_auth.verify_cf_access_token(tok, team_domain=TEAM, aud=AUD, jwks=jwks) ==
"user-123"
67:     {"exp_delta": -10},                # expired
68:     {"aud": "wrong-aud"},              # wrong audience (not this CF app)
69:     {"iss": "https://evil.example"},   # wrong issuer
73: def test_invalid_tokens_rejected(keypair, kw):
76:     edge_auth.verify_cf_access_token(_token(key, **kw), team_domain=TEAM, aud=AUD,
jwks=jwks)
79: def test_bad_signature_rejected(keypair):
80:     """A token signed by a DIFFERENT key (but advertising kid=test-kid) must fail signature
check."""
84:     edge_auth.verify_cf_access_token(_token(other), team_domain=TEAM, aud=AUD, jwks=jwks)
87: def test_alg_none_token_rejected(keypair):
88:     """The canonical 'alg:none' unsigned-token bypass MUST be rejected (algorithms is pinned
RS256)."""
91:     tok = jwt.encode({"email": "e", "aud": AUD, "iss": TEAM, "exp": now + 99, "iat": now},
92:                      key="", algorithm="none", headers={"kid": KID})
94:     edge_auth.verify_cf_access_token(tok, team_domain=TEAM, aud=AUD, jwks=jwks)
97: def test_hs256_algorithm_confusion_rejected(keypair):
98:     """The HS256-confusion bypass (a token claiming alg=HS256, the canonical
asymmetric?symmetric
99:     substitution) MUST be rejected ? algorithms is pinned to RS256, so the verifier refuses
any
100:     non-RS256 alg on the allowlist check (before the key is even used). Hand-crafted because
PyJWT
111:     header = _b64(_json.dumps({"alg": "HS256", "typ": "JWT", "kid": KID}).encode())
112:     payload = _b64(_json.dumps({"email": "e", "aud": AUD, "iss": TEAM, "exp": now +
99}).encode())
113:     tok = (header + b"." + payload + b"." + _b64(b"forged-sig")).decode()
115:     edge_auth.verify_cf_access_token(tok, team_domain=TEAM, aud=AUD, jwks=jwks)
120: def test_resolve_tenant_off_is_noop_even_with_token(keypair):
123:     assert edge_auth.resolve_tenant(
127: def test_resolve_tenant_off_ignores_missing_header():
128:     assert edge_auth.resolve_tenant({}, _cfg(False)) is None
133: def test_resolve_tenant_on_valid(keypair):
137:     assert user == "u@example.com"
140: def test_resolve_tenant_on_missing_header_raises(keypair):
146: def test_resolve_tenant_enabled_but_unconfigured_raises(keypair):
149:     edge_auth.resolve_tenant({edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(True,
team="", aud=""))
152: @pytest.mark.parametrize("team,aud", [((" ", AUD), (TEAM, " ")), ("\\t", "\\n")])
153: def test_resolve_tenant_whitespace_only_config_raises(keypair, team, aud):
154:     """Whitespace-only team/aud counts as UNSET ? same semantics as the boot-time exposure
check
158:     edge_auth.resolve_tenant({edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(True,
team=team, aud=aud))
161: def test_resolve_tenant_header_case_insensitive(keypair):
165:     assert user == "u@example.com"
183:     class _Noop: # don't drain the job (we only assert the created row's owner_id)
192: def test_process_default_off_tags_no_owner(tmp_path, monkeypatch):
195:     below, which asserts a non-NULL owner_id is actually persisted.)"""
200:     assert r.status_code == 202
201:     assert db.get_job(r.json()["job_id"])["owner_id"] is None
204: def test_cors_origins_from_env():
207:     assert m._cors_origins_from_env({}) == ["*"]
208:     assert m._cors_origins_from_env({"RALLY_CORS_ALLOW_ORIGINS": ""}) == ["*"]
209:     assert m._cors_origins_from_env(
214: def test_process_edge_auth_missing_token_401(tmp_path, monkeypatch):

```

15. user (2026-06-21T19:13:22.181Z)

22:- ****Auth at the edge.**** Cloudflare Access gates both hostnames; the engine *also* verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge\_auth\_enabled`), so a direct hit to

```

the tunnel can't spoof identity.
43: pip install -e .[auth]          # PyJWT[crypto] ? REQUIRED when edge auth is on, else every
request 500s
51:   "edge_auth_enabled": true,          // turn the CF-Access JWT verify ON
54:   "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
55:   "cf_access_aud": "<the CF Access application AUD>" // filled in §4
65:> **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails fast** if
`edge_auth_enabled` is on but the `allowed_video_root` jail or `cf_team_domain`/`cf_access_aud`
is missing, and **warns** if `PyJWT` isn't installed. So a half-configured exposure can't
silently come up ? fix what it prints and restart.
96:4. **Copy the application `AUD` tag** ? paste into the engine's `config.json`
`security.cf_access_aud` (§2.2), and set `cf_team_domain` to `https://<your-
team>.cloudflareaccess.com`. Restart the engine.
128:- **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app JWT
verify) only engage then. Do not put the engine behind the tunnel with edge auth off.

```

16. assistant (2026-06-21T19:13:25.547Z)

The claim is fully verified. Let me read the exact §0/§2.2 doc text and confirm the alg-pinning detail (the one potential weakness flagged in the prompt) is genuinely closed, plus check the §4.1 install-extra line. Let me read the doc lines around §0 and §2.2.

17. user (2026-06-21T19:13:26.767Z)

```

18                                     ? async jobs,
SQLite, output/ on disk
19   ``
20
21   - **No inbound ports.** `cloudflared` dials out; the engine stays bound to
`127.0.0.1:8000`. The box's home/public IP is never exposed.
22   - **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also*
verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a
direct hit to the tunnel can't spoof identity.
23   - **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ?
the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine
locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's
already env-driven).
24
25   > **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA
*and* the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see
`LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose
(stable Pages-hosted page + a thin API tunnel).
26
27   ---
28
29   ## 1. Prerequisites

```

18. assistant (2026-06-21T19:13:32.404Z)

I have everything needed. The claim is comprehensively verified against origin/master. Let me confirm the one nuance: the doc says "verifies the Cf-Access JWT in-app (defence in depth)" ? and the code does full RS256 signature + exp + iss + aud verification, with alg pinning (rejecting alg:none and HS256 confusion), JWKS caching with TTL, and a strict no-op when disabled. The header-missing case raises EdgeAuthError ? 401 in main.py.

The only minor caveats worth noting for the owner: (1) the `[auth]` extra must be installed or every request 500s when enabled (the doc DOES call this out at line 43); (2) the in-app verify only guards `/api/process` (the resolve_tenant call site), not necessarily every other endpoint ? but for the alpha that's the work-submitting route.

19. user (2026-06-21T19:13:50.131Z)

Structured output provided successfully

20. assistant (2026-06-21T19:13:58.629Z)

Verified. The §0/§2.2 claim is **confirmed** against ENGINE origin/master.

The engine does real in-app JWT verification (not a header-presence check):

```
`backend/api/auth.py::verify_cf_access_token` runs `jwt.decode(...)` with RS256 pinned,  
`audience=cf_access_aud`, `issuer=cf_team_domain`, and `require=["exp","iss","aud"]`. JWKS is  
fetched from `TTL keyed by team domain. It's a strict no-op when `edge_auth_enabled=false` (returns `None`),  
and on enable a missing/invalid/spoofed token raises `EdgeAuthError` ? 401 at  
`backend/main.py:573` before any work. The two classic bypasses (`alg:none`, HS256 confusion)  
are explicitly tested and rejected in `tests/test_edge_auth.py`.
```

No weaknesses, no discrepancy. Two operational notes (both already in the doc): install the
`[auth]` extra (PyJWT[crypto]) or it 401/500s when enabled; and the verify is wired at the
`/api/process` submit path, so the Cloudflare Access edge gate stays the primary boundary with
the in-app check as the defence-in-depth backstop the doc claims.